こぼさない配膳ロボット

人が行き交うレストランを、強化学習（PPO）と物理に基づく安全層で走る ／ Physical AI 応用1 最終課題 説明資料

結果の要点（学習に使っていない店内配置 各300回）：提案手法の配膳成功率は **96.3%**（3シード平均、標準偏差 0.9 ポイント）。 調整済みの古典手法（ポテンシャル法）は 81.3%、センサを使わない直進制御は 43.3%。 人が常に4人いて物理パラメータの幅を学習時の1.6倍に広げた厳しい条件でも 94.8% を保った。

1. 概要

2輪ロボットの天板に**固定していないコップ**を載せ、テーブル・椅子の配置も歩く人も毎回変わるレストランで、指定されたテーブルの横まで運んで**止まって渡す**タスクを設計した。 コップは MuJoCo 上の剛体なので、急加速・急停止・衝突で実際に倒れる。 LiDAR などのセンサ情報から車輪の速度指令を出す方策を強化学習（PPO）で獲得し、 「残り距離でコップをこぼさずに止まれる速度」を上限とする安全層と組み合わせた。

2. 目的

配膳ロボットはすでに飲食店で使われているが、人の多い店内でトレーの上の飲み物をこぼさず運ぶには、 **①ぶつからない ②こぼさない ③目的地で止まる**を同時に満たす必要がある。 「ぶつからない」だけなら古典的な手法でも実現できるが、物を運ぶ品質（加速度）と、到着の速さ・確実さは互いにぶつかり合う。 本課題の目的は、この複数の要求を**報酬として与えるだけで**、学習によって両立する動き方を獲得できるかを確かめること、 そして**どの工夫が効いているのかを比較実験で数値として示す**ことである。

3. システム構成

要素	内容
シミュレータ	MuJoCo 3.17（物理 200 Hz、制御 20 Hz）。摩擦・接触・慣性を含む。課題で認められている「他のツール」として選択した（理由は付録）
店内	8 m × 6 m。円形テーブル 4〜6卓と椅子をランダム配置。0〜4人が店内を歩く（ロボットが正面にいると立ち止まる）
ロボット	左右独立駆動の2輪（最高 0.8 m/s）。天板にコップ（剛体、質量 0.3 kg、直径 7 cm・高さ 12 cm）
観測（81次元）	LiDAR 36本の距離と前時刻からの変化（人の動きを捉える）／ゴールの距離・方位／速度・角速度／IMU 加速度／直前に実行された指令
行動	左右の車輪の速度指令（連続値）
方策	多層パーセプトロン 256×256、PPO（Stable-Baselines3）、8並列・600万ステップ（1本 約20分、CPUのみ）
安全層	進行方向 ±50° の LiDAR 距離 d から、速度上限 $v_{\max} = \sqrt{(2 \cdot a \cdot (d - 0.05))}$（$a$ = 1.5 m/s²：コップが倒れない減速度）。後退は 0.25 m/s まで
到着判定	ゴールの円（半径 25 cm）内で 0.25 m/s 未満になったら、安全層と同じ減速度で完全に停止させ、停止後もコップが立っていれば配膳成功（学習・評価とも）
報酬	ゴールに近づいた距離 ×3、配膳成功 +10、衝突 −10、こぼす −5、 2 m/s² を超える加速度、人・障害物への接近、後退、安全層の介入量にそれぞれ小さな負の報酬
ランダム化	毎エピソード：店内配置・人数・歩行速度／車体質量 ±25%・コップ質量と摩擦 ±30%・車輪摩擦・モーター特性 ±30%／LiDAR 雑音（σ = 2 cm）と欠測 2%

4. 工夫した点

1. **「こぼす」を物理で起こす。**こぼれ判定を加速度のしきい値で決めるのではなく、コップを剛体として載せた。急ブレーキで本当に前へ倒れることを確かめてから学習に使った（動画冒頭）。
2. **学習と安全層の役割分担。**実際の業務用移動ロボットが LiDAR の安全フィールドで減速・停止するのにならい、「どこへどう曲がるか・どう運ぶか」を学習に、「物理的に止まれる速度」を数式の安全層に任せた。安全層の上限は「コップを倒さない減速度で止まれる速度」から導いており、安全とこぼさないことが矛盾しない。
3. **安全層を「悪用」させない設計。**後退速度の上限を入れた直後、方策は走行時間の 95% を後ろ向きで走り、安全層の上限をなめらかな速度リミッターとして使う抜け道を見つけた（成功率は高いまま）。安全層が指令を修正した量そのものに罰を与え、後退にも罰を与えることでこれを解消した。以後、全手法で「後退していた時間」と「安全層の介入率」を測っている。
4. **公平な比較。**古典手法（ポテンシャル法）は、評価とは別のシードでパラメータを探索して最良の設定にした（`results/pf_tuning.txt`）。さらに古典手法にも同じ安全層を付けた場合を比較し、安全層の効果だけで差がついていないかを確かめた。
5. **アブレーション。**「安全層なし」「コップなしで学習」「物理ランダム化なし」の3種を学習し、各要素が何に効いているかを切り分けた（効果が確認できなかった要素もそのまま報告する）。提案手法は乱数シード3つで学習し、ばらつきも示した。

5. 結果

評価は、学習・チューニングに使っていない店内配置（シード 100000〜100299）での決定的な方策による。

標準条件（学習時と同じ分布）

手法	成功	家具・壁に衝突	人と接触	こぼした	時間切れ	成功時の所要時間
直進制御（センサなし）	43.3%	50.0%	6.7%	0.0%	0.0%	7.2 秒
ポテンシャル法（古典・調整済み）	81.3%	0.0%	0.0%	0.0%	18.7%	10.9 秒
ポテンシャル法＋安全層	81.3%	0.0%	0.0%	0.0%	18.7%	10.9 秒
PPO単体（安全層なし）	85.3%	9.7%	2.0%	3.0%	0.0%	6.3 秒
PPO＋安全層／コップなしで学習	64.7%	0.0%	0.0%	35.0%	0.3%	6.6 秒
PPO＋安全層／物理ランダム化なし	94.7%	0.0%	0.0%	4.3%	1.0%	6.8 秒
提案：PPO＋安全層（3シード平均±標準偏差）	96.3% ±0.9	0.0%	0.0%	2.6%	1.1%	6.9 秒

厳しい条件（常に4人が歩行＋物理パラメータの幅を学習時の1.6倍）

手法	成功	家具・壁に衝突	人と接触	こぼした	時間切れ	成功時の所要時間
直進制御（センサなし）	41.0%	44.7%	14.3%	0.0%	0.0%	6.8 秒
ポテンシャル法（古典・調整済み）	79.7%	0.0%	0.0%	0.0%	20.3%	12.1 秒
ポテンシャル法＋安全層	79.7%	0.0%	0.0%	0.0%	20.3%	12.1 秒
PPO単体（安全層なし）	82.0%	11.7%	4.3%	1.7%	0.3%	6.3 秒
PPO＋安全層／コップなしで学習	65.3%	0.0%	0.0%	34.0%	0.7%	6.8 秒
PPO＋安全層／物理ランダム化なし	93.0%	0.3%	0.0%	6.0%	0.7%	7.1 秒
提案：PPO＋安全層（3シード平均±標準偏差）	94.8% ±1.0	0.1%	0.2%	3.8%	1.1%	7.1 秒

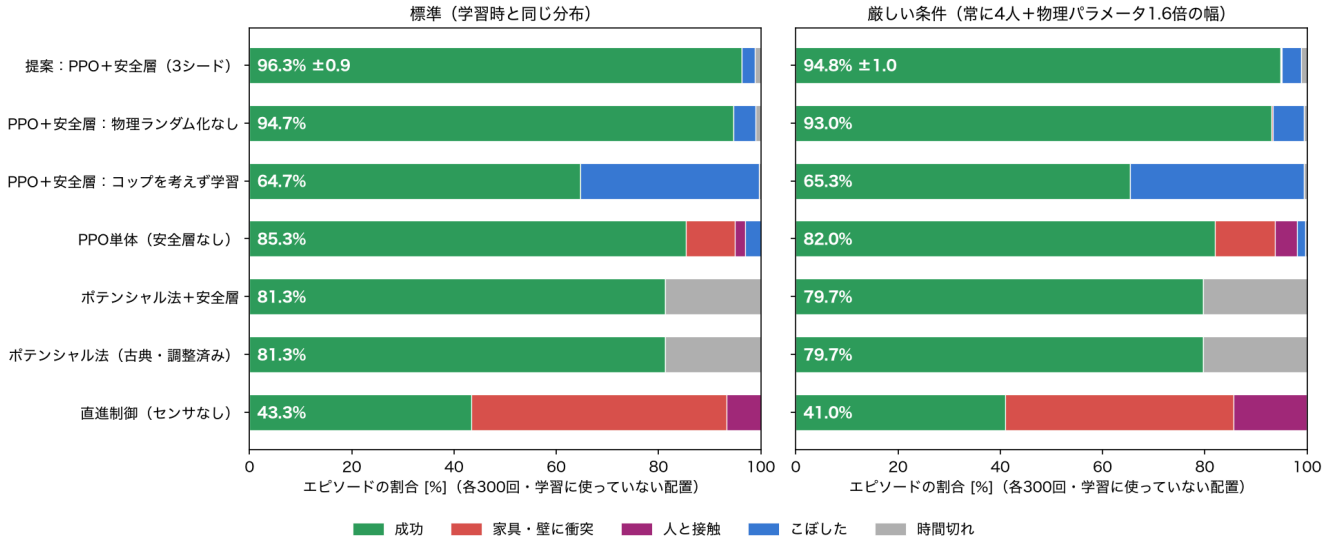


図1 手法ごとの結果の内訳（緑が成功、それ以外は失敗の種類）

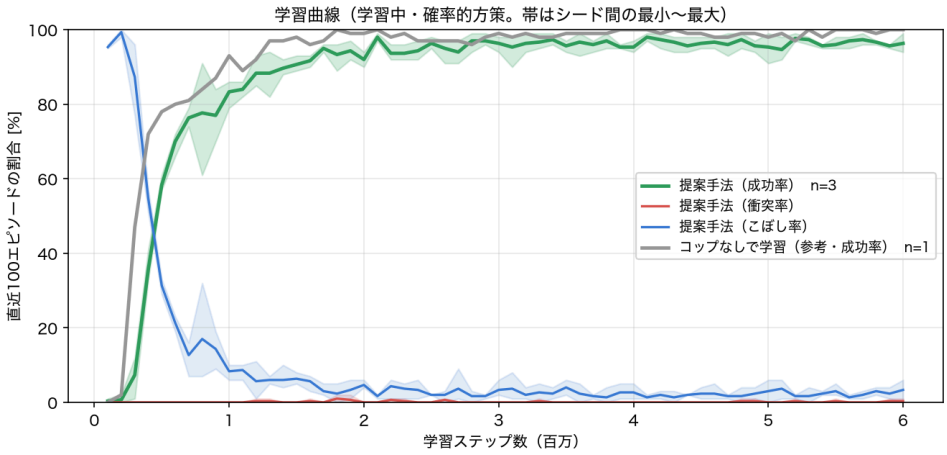


図2 学習曲線（学習中・確率的方策。帯は3シードの最小〜最大）

6. 考察

- **古典手法との比較：**ポテンシャル法は一度もぶつからず、こぼさなかったが、テーブル脇のゴールに近づくと反発力と引力がつり合って立ち往生し、 18.7% が時間切れになった。提案手法は 96.3% で届け、成功時の所要時間も 6.9 秒（ポテンシャル法 10.9 秒）と短い。 一方で、到着直前1秒の最大加速度は提案手法 3.7 m/s²、ポテンシャル法 0.5 m/s² で、提案手法は「速く走って最後にしっかり減速する」運び方をしている。 コップは倒れていないが余裕は小さく、乗り心地の面では古典手法の方が穏やかである。
- **安全層の効果（PPO単体との比較）：**安全層なしで学習・実行した方策は家具・壁への衝突が 9.7%、人との接触が 2.0%（成功 85.3%）。 安全層を付けた提案手法では、それぞれ 0.0%、0.0% で、学習だけでは狭い場所での減速を十分に身につけられないことを、物理に基づく安全層が補っている。 安全層が実際に指令を修正したのは全ステップの 7% で、運転の大部分は学習した方策が行っている。 なお、ポテンシャル法に同じ安全層を付けても結果は変わらなかった（もともと遅く、安全層が働く場面がない）。
- **コップを意識した学習の効果：**コップを載せずに学習した方策は、同じ安全層があってもコップを載せると 35.0% でこぼした（提案手法 2.6%）。 「ぶつからない」ことと「物を運ぶ品質」は別の能力であり、後者は学習時に物理的に体験させる必要がある。
- **物理ランダム化の効果は確認できなかった：**物理パラメータの幅を学習時の1.6倍に広げた厳しい条件で、ランダム化なしで学習した方策の成功は 93.0%、 提案手法は 94.8%（3シードの範囲 93.3〜95.7%）だった。差は乱数シードによるばらつきと同程度で、今回の設定ではランダム化の効果は示せていない。 本環境の速度指令は速度フィードバック付きのモーターで実行されるため、質量や摩擦の変化が動きに表れにくいことが一因と考えている（未検証の推測）。 効果を確かめるには、ランダム化なしのモデルも複数シード学習し、遅延やモーターの飽和など、より実機に近い外乱を加えて比べる必要がある。

- **残る失敗の中身：**提案手法（seed 0）の失敗は主に「こぼす」で、12 件のこぼしのうちすべてが2人以上が歩いている場面だった（0〜1人では 0 件）。 12 件中 5 件では直前 0.5 秒以内に安全層が減速しており、人を避ける急な操作と安全層の急ブレーキの両方が原因になっている。

7. 開発の過程で起きた問題と対処

起きたこと	原因	対処
障害物をすり抜ける（初期版）	MuJoCo の静止体は実行時に動かしても当たり判定が元の位置に残る	動く物はすべて mocap ボディにした
コップが空高く吹き飛ぶ	質量を変えた後に派生定数を再計算していなかった	`mj_setConst` を呼ぶ
静かに走ってもコップがずれていく（30秒で 9 cm）	柔らかい接触によるクリープ	楕円摩擦円錐・`impratio`・ねじれ摩擦（30秒で 1 cm 未満に）
学習した方策がバックで椅子に突っ込む（衝突 約20%）	報酬だけでは狭い場所での減速を学べない	物理に基づく安全層を導入
後ろ向きに走り続ける抜け道	安全層の速度上限を利用	介入量への罰（上記 工夫3）
「配膳完了」の直後にコップが倒れていた	到着の瞬間に成功として打ち切っていたため、方策が全速でゴールに突っ込み急停止していた（判定の穴）	到着後に完全停止させ、コップが立っていることを成功の条件にして全モデルを学習し直した

いずれも「結果が良すぎる・不自然」と感じた時点で、単純な再現実験を作って原因を確かめてから直した。初期版の結果（すり抜けによる見かけの成功）は破棄している。

8. 限界と今後

- シミュレーションのみで、実機では確かめていない。実機では LiDAR の遅延、床の段差、コップの中の液体の揺れ（本研究は剛体で近似）への対応が必要。
- 人は決まった経路を歩き、ロボットが正面にいると止まる単純なモデルである。人の流れの予測や、人がロボットを避ける動きは扱っていない。
- 到着直前の減速が強めで、コップが倒れない範囲ではあるが余裕は小さい。到着前の減速を早める報酬設計（加速度の罰をゴール付近で強めるなど）が次の改善点である。
- 今後：ROS 2 ノードとして方策と安全層を実装し、実機の移動ロボット（例：TurtleBot 系）にトレーを載せて検証したい。 また、注文の自然言語（「窓際のテーブルに水を2つ」）から配膳順を決める上位の計画に言語モデルを使い、本方策を下位の移動に使う構成も考えられる。

付録：使用ツールと再現方法

MuJoCo 3.17 / Gymnasium / Stable-Baselines3 2.9 / PyTorch（CPUのみ、MacBook Air M3）。 ROS 2 + Gazebo ではなく MuJoCo を選んだのは、(1) コップのような小物体の接触・摩擦を細かく設定でき、(2) 1秒に数千ステップの速さで強化学習を回せるためである。 方策の推論は1回 0.07 ミリ秒で、20 Hz 制御に十分間に合う。 コード一式・学習済みモデル・評価結果は GitHub の公開リポジトリにあり、README の手順で再現できる（`evaluate.py` と `make_video.py` は学習なしで実行可能）。 動画は等倍速を基本とし、2倍速の区間と0.4倍速（スロー）の区間は画面右上に明記した。